Embedded Data Structure
(Lecture Note 17)

Sorting Algorithms (4)

2025. 5. 8

Prof. Wonjun Kim

School of Electrical and Electronics Engineering

Review of the Last Class

- **Quick sort : generally provides the optimal solution**
  - **Remember the scheme for quick sort**

    1) Set the pivot (baseline : right-most element)

    2) Roughly sort a given interval with two different indices (i.e., i and j)

    3) Set the new pivot and conduct this task recursively

  - **Quick sort is a kind of type 1 recursion (process→recur1→recur2)**

    : We can convert it to the non-recursive version with stack !

    → However, for this task, two different stack needs to be employed

    → To cope with this limitation, we push and pop the **"index"**

  - **Processing time for quick sort in sorted/reverse cases**

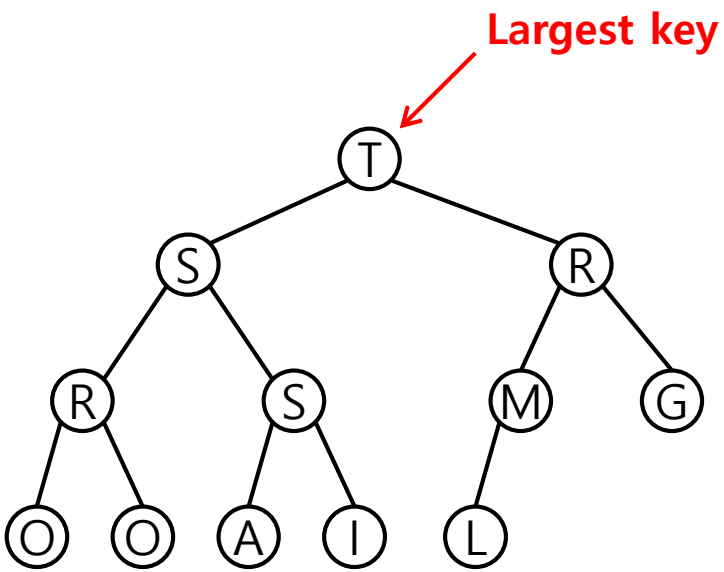
    : Which case is most suitable for quick sort algorithm ?

Heap Sort (1/12)

- **Sorting with priority (a kind of priority queues)**
  - **Implementation with tree structure (using array)**
  - **Heap sort does not require any additional memory**
  - **Complexity : O(NlogN)**

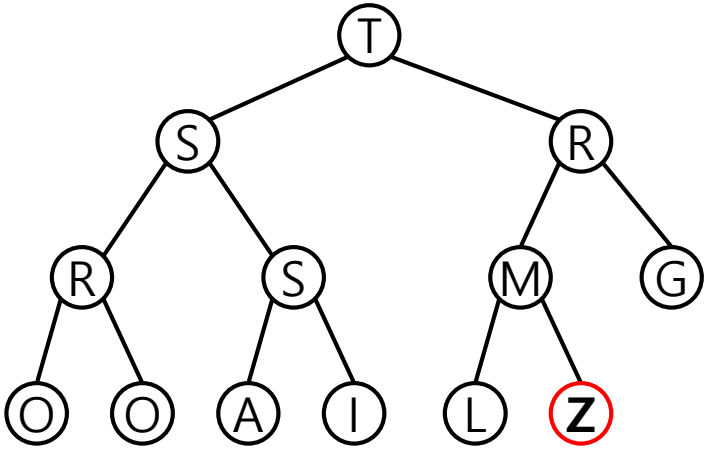
Operations of priority queue :

1. Construct
2. Insert
3. Remove
4. Replace
5. Change
6. Delete
7. Join

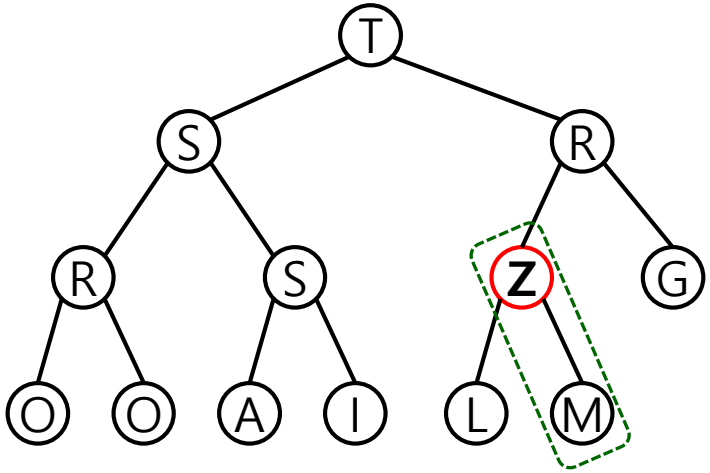


Example of heap ("SORTALGORISM")

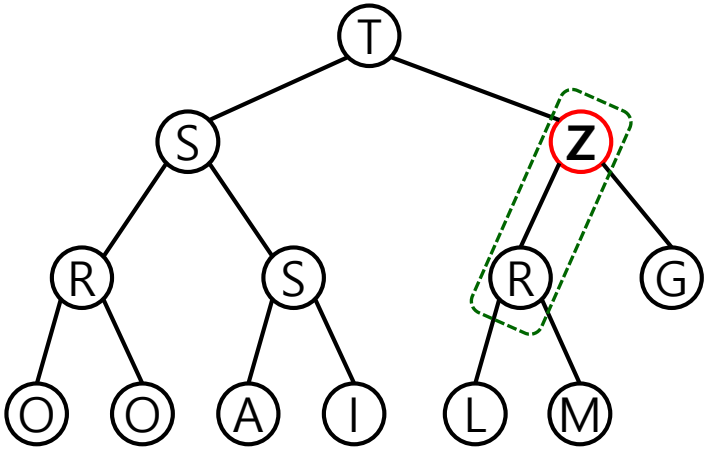
- **Example : Insert (Upheap)**



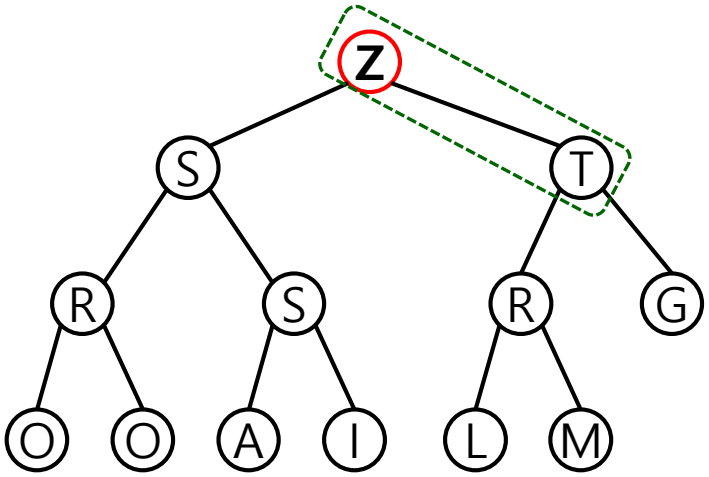
Inset at the end node



Exchange (Z > M)

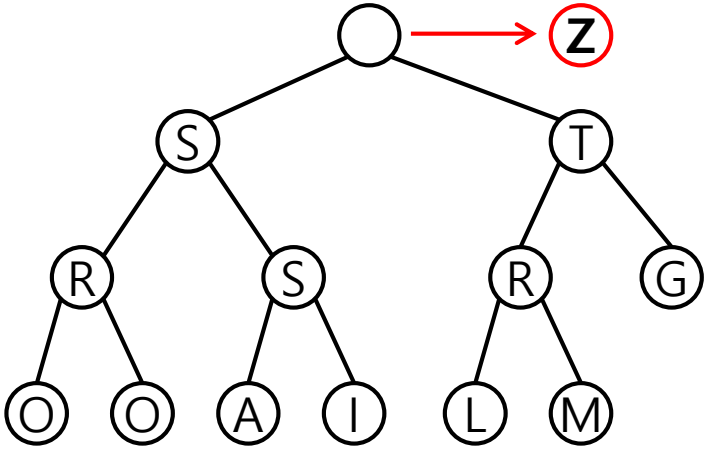


Exchange (Z > R)

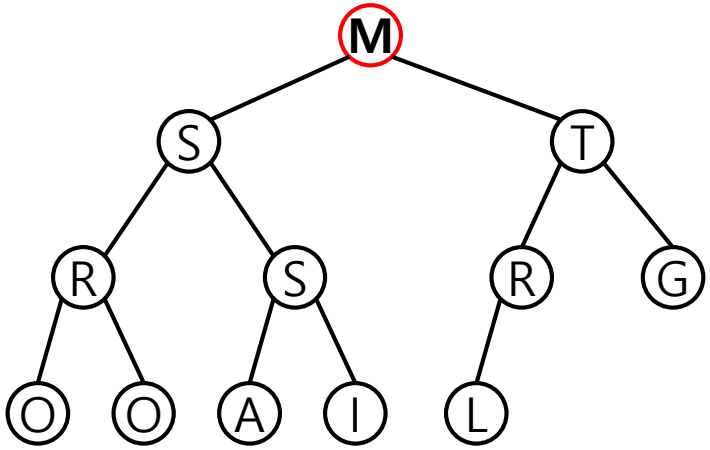


Exchange (Z > T)

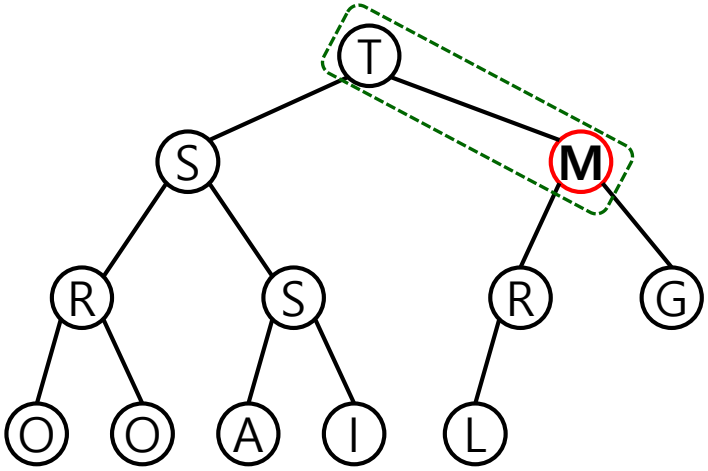
- **Example : Delete (Downheap)**



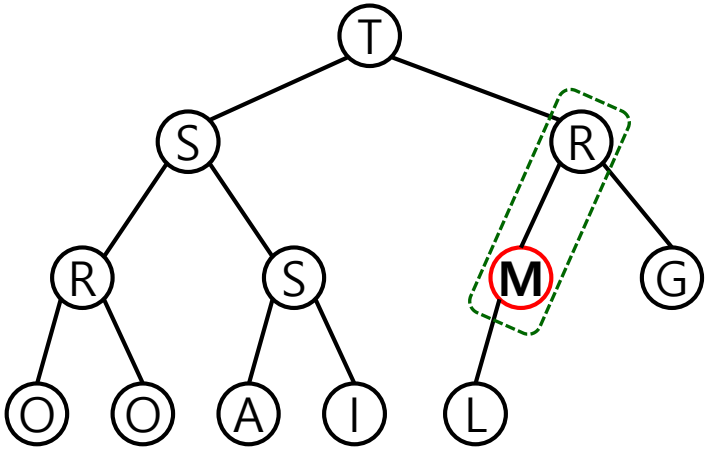
Delete the root



Set the end as the root



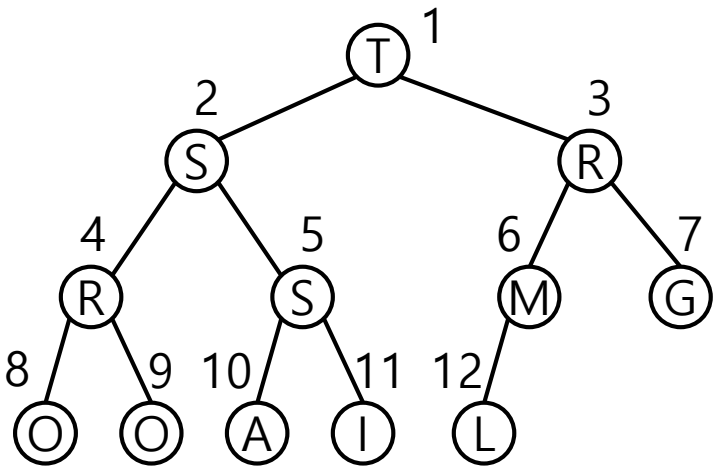
Exchange (T > M)



Exchange (R > M)

Heap sort using array

- **Consider the indexing measure to construct the tree**



Indexing of the heap

Indexing of the heap :

1. Index j is a child of index j/2
2. Parent node j
   → Left child : 2*j
   → Right child : 2*j+1
3. # of internal nodes : N/2

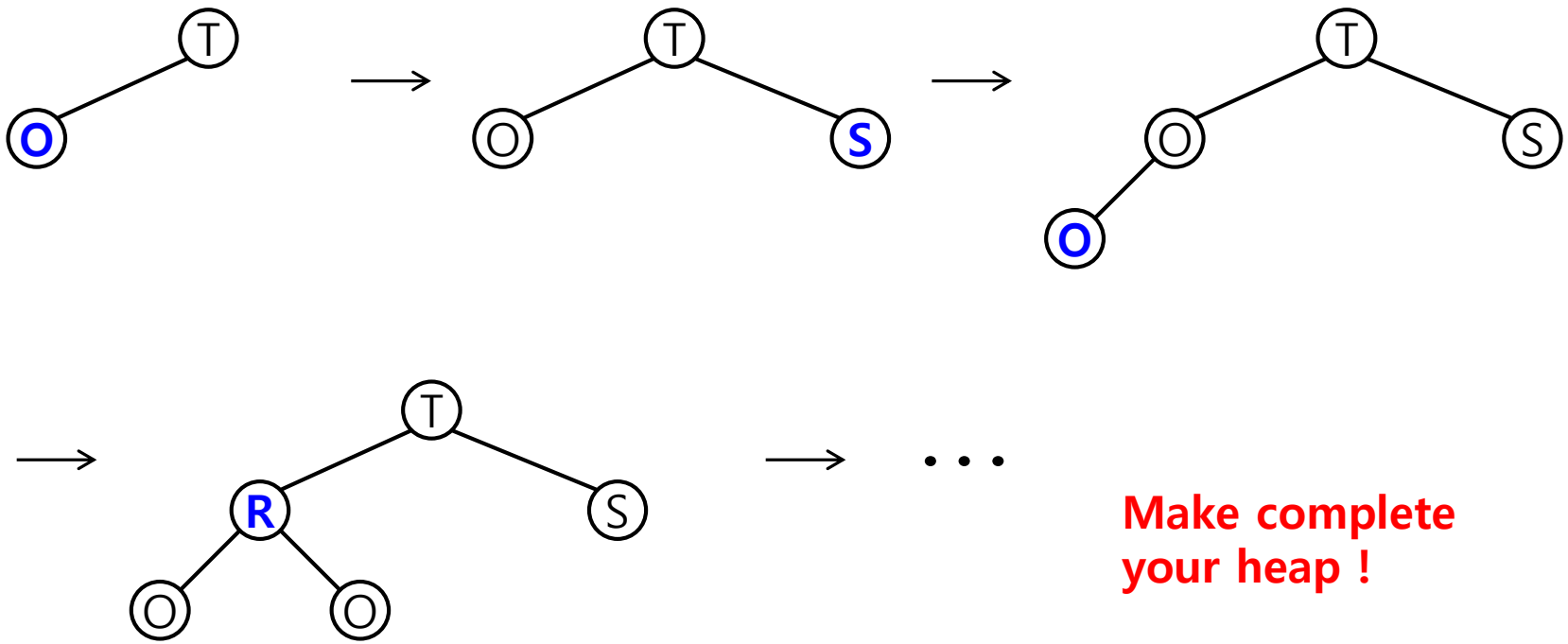
Index	1	2	3	4	5	6	7	8	9	10	11	12
Node	T	S	R	R	S	M	G	O	O	A	I	L

For example,
a[1] = 'T';

Heap Sort (5/12)

- **Top-down heap construction**
  - **Input data : "TOSORTALGORITHM"**

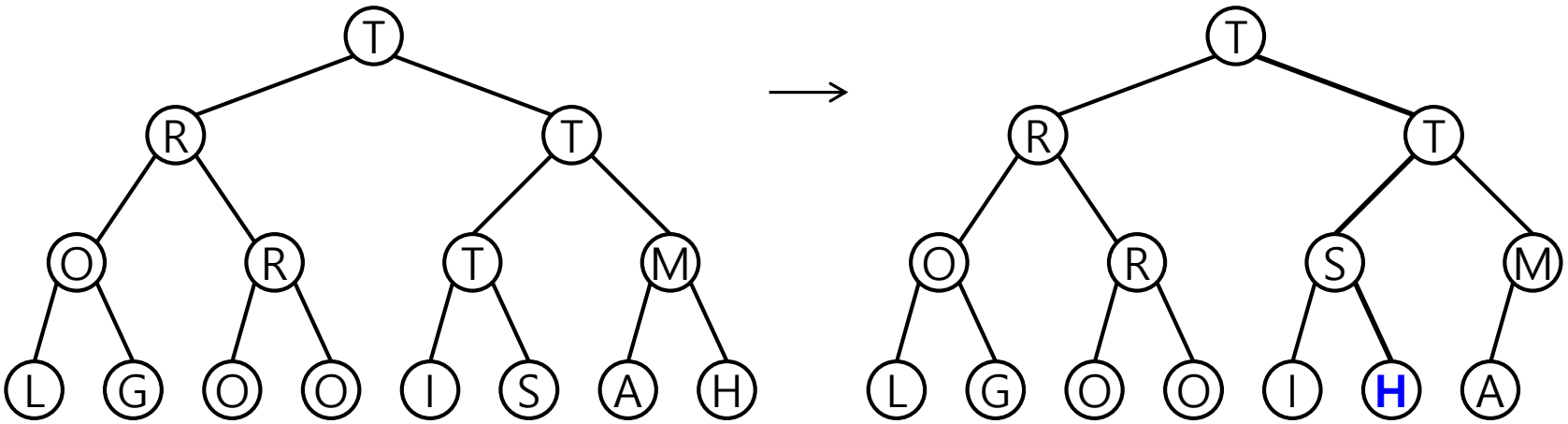
    - Using the "Upheap" procedure (see the following figure)



Make complete your heap !

Heap Sort (6/12)

- **Delete as the sorting tool**
  - **Extract the smallest value and then tree is arranged**
    - Store the root node (i.e., max) at each step



Draw the remaining procedure and

Check your sorting result !

Heap Sort (7/12)

▪ Implementation : Upheap & Downheap

- **For construction and sorting (start with a[1])**

```
void upheap(int *a, int k)
{
    int v;
    v = a[k];
    a[0] = INT_MAX;
    while(a[k/2] <= v){
        a[k] = a[k/2];
        k /= 2;
    }
    a[k] = v;
}
```

```
void downheap(int *a, int N, int k)
{
    int i, v;
    v = a[k];
    while(k<=N/2){
        i = k<<1;
        if(i < N && a[i] < a[i+1]) i++;
        if(v >= a[i]) break;
        a[k] = a[i];
        k = i;
    }
    a[k] = v;
}
```

KU KONKUK UNIVERSITY

Implementation : other functions

- **For construction and sorting (start with a[1]) – cont'd**

```
void insert(int *a, int *N, int v)
{
    a[++(*N)] = v;
    upheap(a, *N);
}
```

Why pointer ?

```
void extract(int *a, int *N)
{
    int v = a[1];
    a[1] = a[(*N)--];
    downheap(a, *N, 1);

    return v;
}
```

Final sorting function

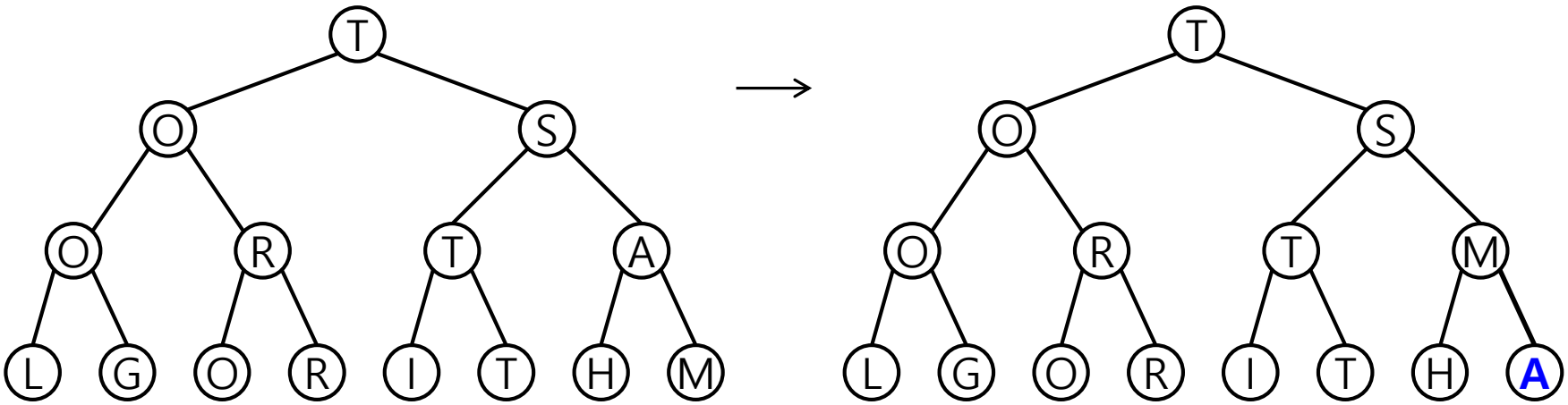
```
void heap_sort(int *a, int N)
{
    int i;
    int hn = 0; // # of heap nodes

    for(i = 1; i<=N; i++)
        insert(a, &hn, a[i]);

    for(i = hn; i>=1; i--)
        a[i] = extract(a, &hn);
}
```

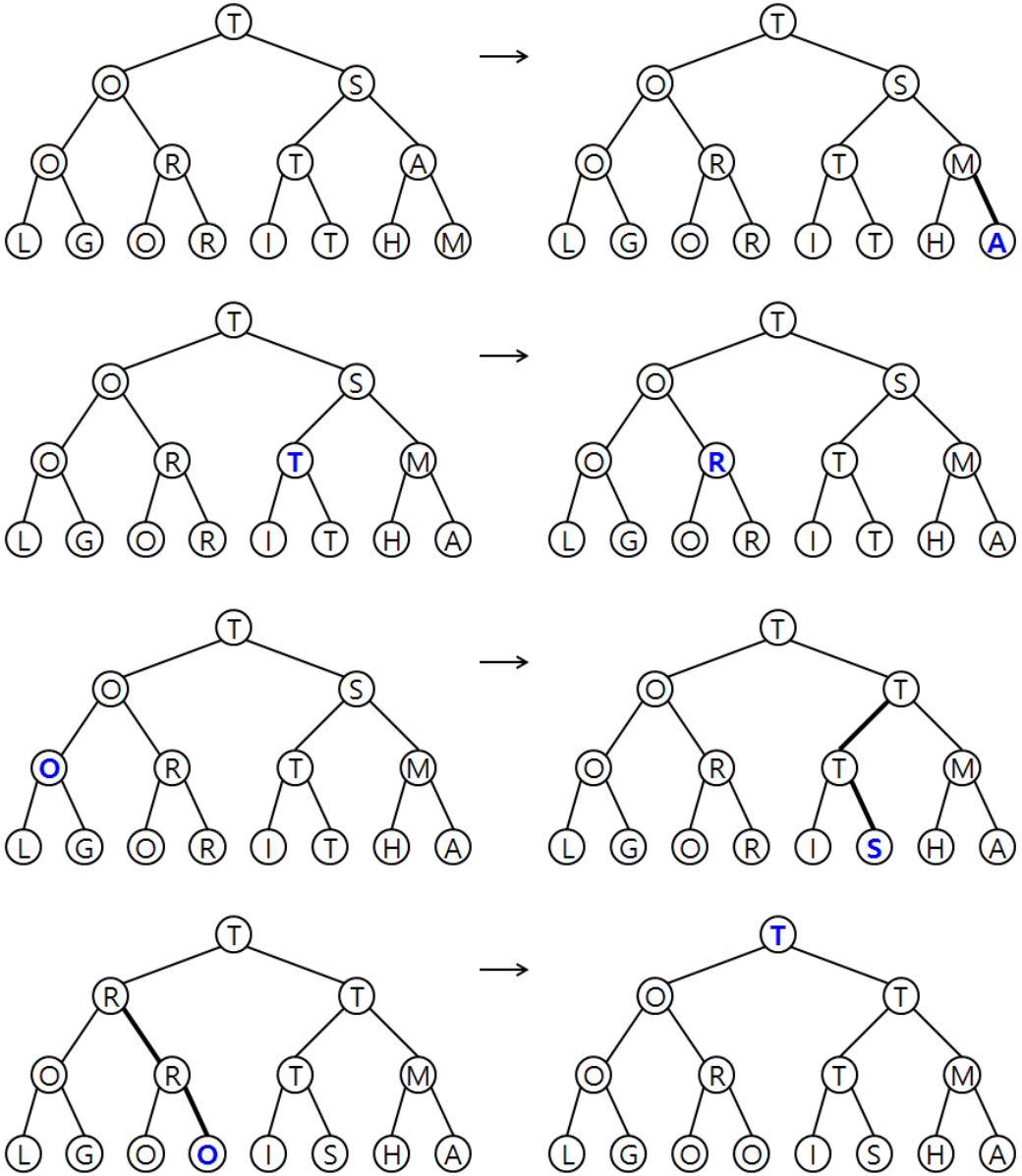
Heap Sort (9/12)

- **Bottom-up heap construction**
  - **Make the binary tree using input data in a sequential way**
  - **Start with the end of internal nodes and conduct downheap**



→ • • • ➡ **Draw the remaining procedure**

Heap Sort (10/12)



15

Heap Sort (11/12)

- **Implementation (yourself)**
  - **Function for the bottom-up heap sorting**

```
void heap_sort(int *a, int N)
{
    int k, t;

    for( [                    ] )
        downheap(a, N, k);

    while(N>1){
        [                    ]
        [                    ]
        a[N] = t;
        [                    ]
    }
}
```

Loop for downheap

Root ← max value

Assign the value of end to the root

N-- and then conduct downheap

Heap Sort (12/12)

- **Improvement of processing time**

Top-down

N	Sorted	Random	Half-sorted	Reverse
100	0	1	1	1
500	2	2	2	2
1000	6	5	5	3
2000	12	9	11	8
5000	36	24	31	22
7000	52	34	44	32
10000	78	51	65	47

Bottom-up

N	Sorted	Random	Half-sorted	Reverse
100	1	0	0	0
500	2	1	2	2
1000	3	4	3	3
2000	8	7	8	7
5000	21	20	20	19
7000	29	29	29	27
10000	44	43	43	41

Summary

- **Heap sort : tree-based sorting algorithm**
  - **Remember the up and down heap for sorting**
  - **Remember top-down and bottom-up construction schemes**